Nezha privacy 贡献指南

本文档详细记录了为 nezha-privacy/PPPU 项目贡献代码的完整流程。

前置准备

1. 配置 Git 身份信息（仅当前仓库生效）

在仓库目录下使用 `--local` 配置，不会影响全局或其他仓库的 Git 配置：

Code block

```
cd PPPU
git config --local user.name "你的GitHub用户名"
git config --local user.email "你的GitHub邮箱"
```

2. 配置 SSH Key（如果尚未配置）

生成专门用于私人 GitHub 账号的 SSH key，与公司 / 其他账号互不干扰：

Code block

```
# 生成新 key（不覆盖已有 key）
ssh-keygen -t ed25519 -C "你的邮箱" -f ~/.ssh/id_ed25519_personal

# 查看公钥并复制
cat ~/.ssh/id_ed25519_personal.pub
```

在 `~/.ssh/config` 中添加配置：

Code block

```
Host github.com
    IdentityFile ~/.ssh/id_ed25519_personal
    User git
```

将公钥添加到 GitHub：

1. 打开 https://github.com/settings/keys
2. 点击 **New SSH key**

3. 粘贴公钥内容并保存

验证连接：

```
ssh -T git@github.com
# 预期输出：Hi <你的用户名>! You've successfully authenticated...
```

3. Fork 仓库

1. 打开 https://github.com/nezha-privacy/PPPU
2. 点击右上角 **Fork** 按钮，将仓库 fork 到自己的账号下

4. Clone 你的 Fork

```
git clone git@github.com:<你的用户名>/PPPU.git
cd PPPU
```

5. 添加 upstream remote

```
git remote add upstream git@github.com:nezha-privacy/PPPU.git
```

验证 remote 配置：

```
git remote -v
# origin    git@github.com:<你的用户名>/PPPU.git (fetch)
# origin    git@github.com:<你的用户名>/PPPU.git (push)
# upstream  git@github.com:nezha-privacy/PPPU.git (fetch)
# upstream  git@github.com:nezha-privacy/PPPU.git (push)
```

开发流程

1. 同步上游最新代码

每次开发新功能前，先同步 upstream 的最新代码：

Code block

```
git fetch upstream
git checkout main
git rebase upstream/main
```

2. 创建功能分支

Code block

```
git checkout -b <分支名>
# 例如: git checkout -b fix-network-bug
```

分支命名建议：

- `feat/xxx` — 新功能
- `fix/xxx` — Bug 修复
- `docs/xxx` — 文档更新
- `refactor/xxx` — 代码重构

3. 开发与提交

Code block

```
# 修改代码后，查看变更
git status
git diff

# 暂存并提交
git add <文件>
git commit -m "简洁描述本次变更"
```

Commit message 建议格式：

- `feat: 添加 xxx 功能`
- `fix: 修复 xxx 问题`
- `docs: 更新 xxx 文档`
- `refactor: 重构 xxx 模块`

4. 推送到你的 Fork

Code block

```
git push -u origin <分支名>
```

提交 PR（Pull Request）

1. 创建 PR

通过 `gh` CLI 创建：

Code block

```
gh pr create --repo nezha-privacy/PPPU \
  --title "PR 标题" \
  --body "## 概要
- 变更内容描述
"
```

或者在浏览器中操作：

1. 打开 https://github.com/nezha-privacy/PPPU
2. 点击 **Compare & pull request**
3. 填写标题和描述
4. 点击 **Create pull request**

2. 请求 Review

> ⚠ **重要：PR 提交后，务必拉 `zcbbb266` 进行 Code Review。通过 Review 后再 Merge。**

在 PR 页面右侧 **Reviewers** 栏添加 `zcbbb266`，或使用命令行：

Code block

```
gh pr edit <PR编号> --add-reviewer zcbbb266
```

3. 根据 Review 意见修改

如果 Reviewer 提出修改意见：

Code block

```
# 在同一分支上继续修改
```

```
git add <修改的文件>
git commit -m "fix: 根据 review 意见修改 xxx"
git push origin <分支名>
```

Push 后 PR 会自动更新，无需重新创建。

4. Merge

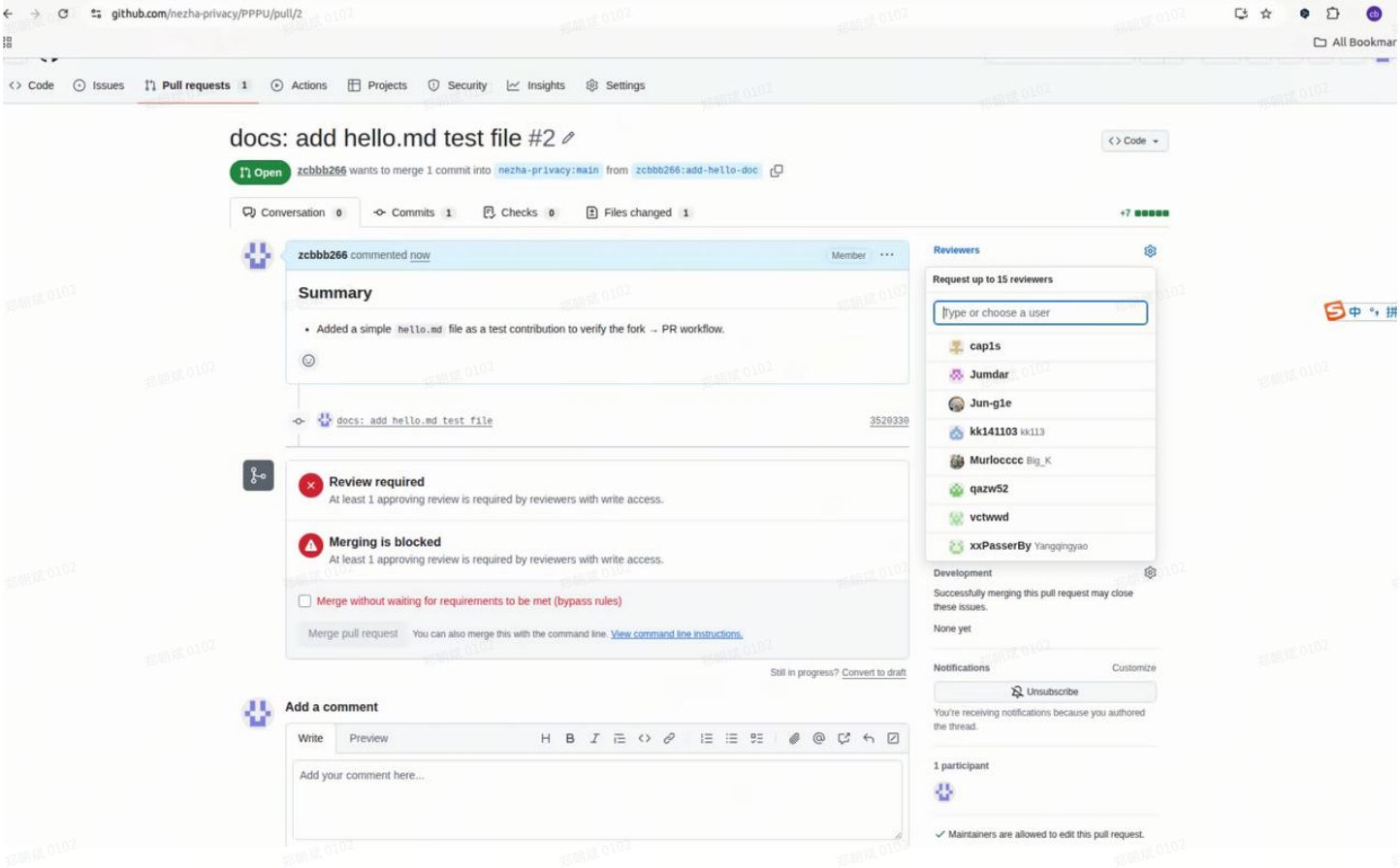
Review 通过后，由 Reviewer 或有权限的人点击 **Merge pull request** 完成合并。

常用命令速查

操作	命令
同步上游代码	`git fetch upstream && git rebase upstream/main`
创建分支	`git checkout -b <分支名>`
查看状态	`git status`
暂存所有变更	`git add .`
提交	`git commit -m "描述"`
推送	`git push origin <分支名>`
创建 PR	`gh pr create --repo nezha-privacy/PPPU`
添加 Reviewer	`gh pr edit <PR号> --add-reviewer zcbbb266`

注意事项

1. **不要直接向 `main` 分支提交代码**，始终通过功能分支 + PR 的方式。
2. **每次开发前先同步 upstream**，避免合并冲突。
3. **PR 提交后必须拉 `zcbbb266` Review**，Review 通过后再 Merge。
4. **一个 PR 只做一件事**，保持变更范围清晰。



使用 OpenCode 辅助开发隐私算法

以下章节介绍如何借助 OpenCode（终端 AI 编程助手）配合 PPPU 的开发文档，快速构建上层隐私计算算法。

第一章：安装与配置 OpenCode

1.1 安装 OpenCode

方式一：一键安装脚本（推荐）

Code block

```
curl -fsSL https://raw.githubusercontent.com/opencode-ai/opencode/refs/heads/main/install | bash
```

方式二：Homebrew（macOS / Linux）

Code block

```
brew install opencode-ai/tap/opencode
```

方式三：Go 安装

Code block

```
go install github.com/opencode-ai/opencode@latest
```

安装完成后验证：

Code block

```
opencode --version
```

1.2 配置 AI Provider

OpenCode 需要至少一个 AI Provider 的 API Key。在项目根目录或 `~/.config/opencode/` 下创建 `.opencode.json`：

Code block

```
{
  "providers": {
    "anthropic": {
      "apiKey": "你的 Anthropic API Key"
    }
  },
  "agents": {
    "coder": {
      "model": "claude-sonnet-4-20250514",
      "maxTokens": 16000
    }
  }
}
```

支持的 Provider 包括：OpenAI、Anthropic Claude、Google Gemini、GitHub Copilot、Groq、Azure OpenAI 等。也可以通过环境变量配置：

Code block

```
export ANTHROPIC_API_KEY="你的Key"
# 或
export OPENAI_API_KEY="你的Key"
# 或
export GITHUB_TOKEN="你的Token"  # GitHub Copilot
```

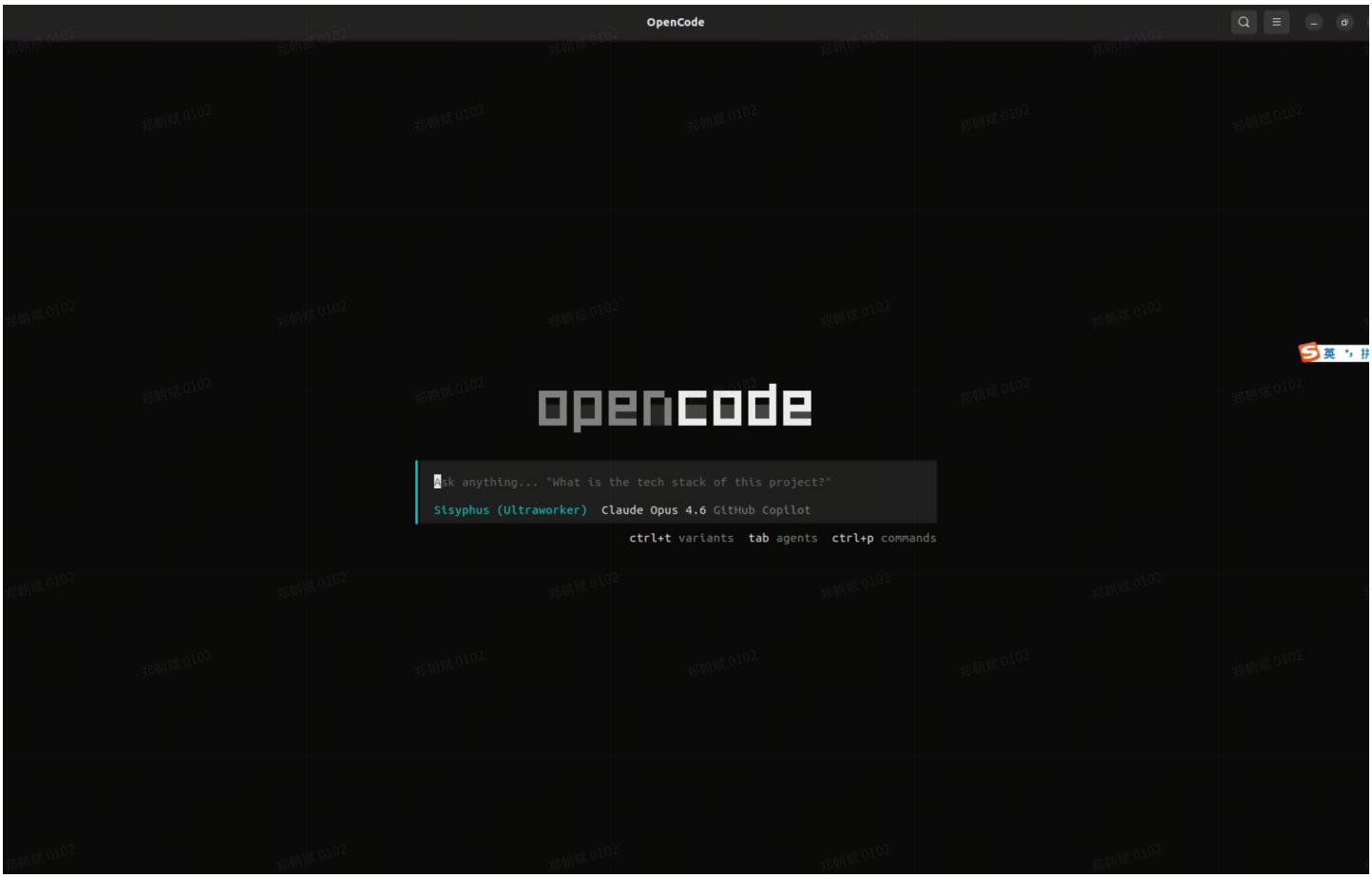
1.3 启动 OpenCode

在项目目录下启动：

Code block

```
cd /path/to/your/project
opencode
```

OpenCode 会自动识别当前目录下的代码结构，提供上下文感知的编程辅助。



第二章：配置 PPPU 开发 Skill

OpenCode 的 Skill 机制允许将 PPPU 的开发文档注入 AI 的上下文中，使 AI 理解 PPPU 的 API 和开发模式，从而能直接帮你编写基于 PPPU 的隐私算法。（注：可以让opencode读取/pppu/doc/development下的开发文档并自动配置skills,提示词：**pppu是一个底层的安全算子，其开发文档在doc/development下，请你配置成skills。然后就可以用pppu去开发新的上层算法。**）

```
# Initialize pppu skill

$ python3 /home/chaobinzheng/.config/opencode/skills/skill-creator/scripts/init_skill.py pppu --path /home/chaobinzheng/.config/opencode/skills/ 2>&1

🚀 Initializing skill: pppu
   Location: /home/chaobinzheng/.config/opencode/skills/

✅ Created skill directory: /home/chaobinzheng/.config/opencode/skills/pppu
✅ Created SKILL.md
✅ Created scripts/example.py
✅ Created references/api_reference.md
✅ Created assets/example_asset.txt

✅ Skill 'pppu' initialized successfully at /home/chaobinzheng/.config/opencode/skills/pppu
…
Click to expand
```

2.1 初始化 Skill

Code block

```
# 进入 OpenCode skills 目录
mkdir -p ~/.config/opencode/skills/pppu/references
```

2.2 创建 SKILL.md

在 `~/.config/opencode/skills/pppu/SKILL.md` 中编写 Skill 定义。这是 Skill 的入口文件，告诉 AI 何时激活、如何使用 PPPU：

Code block

```
---
name: pppu
description: "PPPU secure MPC computation framework. Use for developing
  upper-layer privacy-preserving algorithms using Value/Context API with
  arithmetic, comparison, math, and shape operations over the Semi2k protocol.
  Triggers: 'pppu', 'secure computation', 'MPC algorithm', 'privacy preserving',
  'secret sharing'. Source: /path/to/PPPU"
---

# PPPU — Secure Computation Skill

## Overview

PPPU is a C++ secure multi-party computation (MPC) framework...
（后续内容参考 doc/development/ 和 doc/reference/ 中的 API 文档）
```

SKILL.md 应包含以下核心内容：

节	内容
Architecture	PPPU 分层架构：Upper-Layer → Context → MPC → Data → Network

Key Concepts	Value 类型、Visibility 状态、Context 对象、数据流
Algorithm Pattern	标准开发模板（includes → setup → private → input → compute → open → decode）
Operations Reference	算术 / 比较 / 数学 / 形状操作的速查表
Build Integration	CMakeLists.txt 集成方式
Constraints	C++20、GCC 13+、Party 启动顺序、fracbits 等

2.3 添加 API Reference 文件

将 PPPU 的详细 API 文档放到 `references/` 目录下，供 AI 在需要时查阅：

Code block

```
~/.config/opencode/skills/pppu/
├── SKILL.md                        # Skill 入口（架构 + 速查）
└── references/
    ├── context_api.md              # Context 层完整 API（Value、Factory、Basic、Compare、Math、Shape）
    └── low_level_api.md            # 底层 API（NDArray、Z2k/Zp、Serialization、Network）
```

`references/context_api.md` 的内容来源于 PPPU 仓库中的：

- `doc/development/` — 开发文档
- `doc/reference/context.md` — Context API 参考
- `src/context/` — 源码中的头文件

`references/low_level_api.md` 的内容来源于：

- `doc/reference/ndarray.md`、`doc/reference/datatypes.md` 等
- `doc/development/ndarray.md`、`doc/development/serialization.md` 等

2.4 验证 Skill

Code block

```
# 如果安装了 skill-creator 的验证脚本
python3 ~/.config/opencode/skills/skill-creator/scripts/quick_validate.py  \
    ~/.config/opencode/skills/pppu

# 预期输出：Skill is valid!
```

配置完成后，在 OpenCode 中提到 "pppu"、"secure computation"、"MPC algorithm" 等关键词时，AI 会自动加载 PPPU 的 API 知识。

第三章：使用 OpenCode 开发上层算法（以 SecureRJ 为例）

SecureRJ 是基于安全多方计算的近似查询框架。以下展示如何用 OpenCode + PPPU Skill 高效开发此类上层算法。

3.1 克隆并配置项目

Code block

```
# Fork nezha-privacy/SecureRJ 到自己账号后 clone
git clone git@github.com:<你的用户名>/SecureRJ.git
cd SecureRJ

# 配置 Git 身份（仅本仓库生效）
git config --local user.name "你的用户名"
git config --local user.email "你的邮箱"

# 添加 upstream
git remote add upstream git@github.com:nezha-privacy/SecureRJ.git
```

3.2 在项目中启动 OpenCode

Code block

```
cd SecureRJ
opencode
```

3.3 让 AI 帮你编写隐私算法

在 OpenCode 的对话中，你可以直接描述需求，AI 会基于 PPPU Skill 中的知识来编写代码。

示例对话 1：实现安全求和

Code block

```
> 用 PPPU 实现一个两方安全求和算法。每方各持有一个 int 向量，
> 计算向量元素之和后 reveal 结果。写到 src/secure_sum.cpp。
```

AI 会自动使用正确的 PPPU API（`make_private` → `input` → `add` → `open` → `decode`），生成完整可编译的代码。

示例对话 2：实现安全比较

Code block

```
> 在 SecureRJ 里添加一个安全比较函数：两方各输入一个向量，
> 输出逐元素的最小值。使用 PPPU 的 compare 操作。
```

示例对话 3：修改 CMakeLists.txt

Code block

```
> 帮我把 secure_sum.cpp 加到 CMakeLists.txt 里，
> 链接 PPPU 库和所有必要依赖。
```

3.4 开发工作流

Code block

```
┌──────────────────────────────────────────────────────────┐
│  1. 在 OpenCode 中描述算法需求                            │
│  2. AI 基于 PPPU Skill 生成代码                           │
│  3. 检查生成的代码，必要时让 AI 修改                       │
│  4. 编译测试：mkdir build && cd build && cmake .. && make │
│  5. 运行测试：./program 0 2 &  ./program 1 2              │
│  6. 提交代码并创建 PR（参考前面的贡献流程）                 │
└──────────────────────────────────────────────────────────┘
```

3.5 提交贡献

开发完成后，按照本文档前面的开发流程和提交 PR章节操作：

Code block

```
git checkout -b feat/secure-sum
git add src/secure_sum.cpp CMakeLists.txt
git commit -m "feat: add secure sum algorithm using PPPU"
git push -u origin feat/secure-sum
gh pr create --repo nezha-privacy/SecureRJ \
  --title "feat: add secure sum algorithm" \
  --body "## 概要
- 基于 PPPU Context API 实现两方安全求和
- 添加 CMake 构建配置
"
gh pr edit <PR号> --add-reviewer zcbbb266
```

> ⚠ 提醒：PR 提交后务必拉 `zcbbb266` Review，通过后再 Merge。